A Scalar Parity Refinement of the Subexponential SAT Counter

Proved arithmetic improvements and a conditional route to a better running-time exponent

Research note prepared for Vladimir Reshetnikov

October 26, 2026

Abstract

We examine the October 25 revision of OpenAI's preprint *The Exponential Time Hypothesis is False*, pinned to the repository revision specified below. We obtain an elementary, exact replacement for its dense-query parity lift. The replacement removes the truncated polynomial variable V, uses scalar residues, and improves the retained incidence-degree cap from $k(2n+a)$ to $k(n+a)$ for a sparse degree-k tests. Its correctness, coefficient divisibility, and sharpness are proved and checked by executable finite tests. This gives precise improvements in arithmetic representation and local operations, but preserves the manuscript's leading general-input exponent $O_k(n \log\log n/\sqrt{\log n})$.

For a stronger improvement we propose applying contraction a second time to a compact encoding of its mode sum. We derive the conditional bound $(n+m)^{O_k(1)}2^{O_k(n(\log\log n)^{3/2}/\log n)}$, including the necessary precision analysis and retuning of two preprocessing stages. The missing step is an explicit compact factorization of the resulting high-degree group tensors. We state that step as an unproved assumption and explain why neither the existing theorem nor index compression supplies it. Thus this note does not claim a newly proved improved leading worst-case SAT exponent.

Status of the requested breakthrough. A concrete arithmetic refinement is proved. A stronger asymptotic algorithm is proposed conditionally. The structural factorization needed to turn that proposal into a theorem has not been constructed. The included Python programs validate the finite algebra and indexing identities; they are not complete subexponential SAT solvers.

Contents

1 Result, baseline, and scope **3**
- 1.1 The comparison that matters . . . 3
- 1.2 Source provenance and validation boundaries . . . 3

2 Locating the two asymptotic losses **4**

3 An exact scalar parity replacement **4**
- 3.1 Normalization and the local indicator . . . 4
- 3.2 Coefficientwise divisibility . . . 5

4 Integrating the replacement into the exact counter **6**
- 4.1 The counting identity . . . 6
- 4.2 Coefficient evaluation without expanding the lift . . . 7

5 Precisely what becomes faster or smaller **7**

6 Why several immediate exponent improvements fail **8**
- 6.1 Incidence-support indexing has a real entropy cost . . . 8
- 6.2 The weighted-partition estimate is sharp for its relaxation . . . 8
- 6.3 Nonnegative Hermite tapers have a restricted limitation . . . 9
- 6.4 Other changes that do not certify a faster bound . . . 9

7 The stronger proposal: contract the mode sum again **9**
- 7.1 One deformation stage with more groups . . . 10
- 7.2 Exact pair budgets and a shorter index network . . . 10
- 7.3 The missing structural result . . . 11

8 Conditional running time and all remaining costs **12**
- 8.1 A modest extension of the existing contraction analysis . . . 12
- 8.2 The conditional contraction theorem . . . 13
- 8.3 Retuning preprocessing is necessary . . . 13

9 A narrower alternative: aggregate equal variable unions **14**

10 Implementation artifacts and checks **14**
- 10.1 Scalar parity prototype . . . 14
- 10.2 Weighted-mode indexing prototype . . . 15
- 10.3 What was and was not validated . . . 15

11 Recommended next mathematical step **16**

A Exact weighted-tuple indexing **16**

B Source locations for an implementation patch **16**

1 Result, baseline, and scope

1.1 The comparison that matters

Let n be the number of declared variables, m the number of input clauses, and k a fixed clause width. The user's case is $k = 3$. Put $\lambda = \lceil \log_2 n \rceil$ and $\nu = \lceil \log_2 \lambda \rceil$ when n is sufficiently large. All asymptotic statements allow an explicit exhaustive fallback on a bounded range of smaller sizes. Logarithm bases affect constants only.

The manuscript states a deterministic exact counter with polynomial space and the first two bounds in Table 1. These are stronger than the bound described on the repository's Lean scope page. The latter is $2^{O_k(n(\log\log n)^2/\sqrt{\log n})}$ times a polynomial in encoded input length [1, 4]. This distinction matters: removing one $\log\log n$ from that weaker description would not improve the revised manuscript.

Method or setting	Exponent, omitting polynomial factors	Status here
Source, arbitrary density	$O_k(n \log\log n/\sqrt{\log n})$	Inherited manuscript theorem
Source, $m \le Dn$	$O_{k,D}(n/\sqrt{\log n})$	Inherited manuscript theorem
Scalar parity replacement	$O_k(n \log\log n/\sqrt{\log n})$	Proved replacement; same leading order
Second contraction, arbitrary density	$O_k(n(\log\log n)^{3/2}/\log n)$	Conditional on Assumption 7.1
Second contraction, $m \le Dn$	$O_{k,D}(n\sqrt{\log\log n}/\log n)$	Conditional on Assumption 7.1
Union-support compiler	$O_k(n/\sqrt{\log n})$	Separate unproved route

Table 1: Every algorithm in the table would be deterministic and use polynomial space under its stated assumptions. A conditional row is not an established improvement.

The proposed second-contraction exponent is strictly smaller in asymptotic order: the ratio to the manuscript's arbitrary-density exponent is

$$\frac{n(\log\log n)^{3/2}/\log n}{n\log\log n/\sqrt{\log n}} = \sqrt{\frac{\log\log n}{\log n}} \longrightarrow 0.$$

This comparison concerns the bounding functions. It is neither an observed speedup nor a proof that the unproved factorization exists.

1.2 Source provenance and validation boundaries

The inspected repository commit is

`7ef5c0cc5ca4b0e072b7a050fca09ab24bc6fa3a`.

The principal files are the main manuscript, its quantitative reduction, and its quantitative contraction proof [1, 2, 3]. A machine-readable manifest in the package records the exact retrieved URLs and hashes. We read the algorithms and their parameter arguments, rather than assuming the title alone establishes their correctness.

The elementary algebra in Sections 3–5 is proved independently below. Claims that it gives a complete subexponential counter invoke the source contraction and reduction theorems. We did not

rebuild the complete Lean formalization, implement the full contraction engine, or independently verify every theorem in the preprint. The repository's comparator statement is a specification; it should not be confused with an executed proof check. No claim of priority is made for the familiar parity indicator identity itself.

2 Locating the two asymptotic losses

The quantitative reduction first obtains systems of $a = O_k(n)$ polynomial equations over $\mathbb{F}_2$. Each equation has bounded degree and $(\log n)^{O_k(1)}$ listed monomials. It gives every test–variable incidence its own Boolean indeterminate. If N is the total number of these incidences, then $N \le n(\log n)^{O_k(1)}$.

Its polynomial lift has surviving total incidence degree $K = O_k(n)$. After grouping tests and variables, it enumerates degree allocations and encodes selected incidence subsets by binary indices. This is the construction labeled `eq:sharp-wire-budget` in the source [2]. Its size estimate is

$$M \le J^2 + \log_2 \binom{N+K}{K} \le J^2 + K \log_2 \frac{\mathrm{e}(N+K)}{K} = O_k(n \log\log n), \qquad J = \left\lceil n^{1/4} \right\rceil. \tag{1}$$

Our first refinement changes K and the coefficient ring, but leaves the order of this bound unchanged.

The second loss is inside the contraction engine. With M active wires and u retained valuation levels, the source has at most $O(M/D)$ modes of each positive weight and a working precision $P = O(MD/u)$. Counting weighted mode tuples gives

$$\exp\left(O\sqrt{\frac{M}{D}\,P}\right) = \exp(O(M/\sqrt{u})). \tag{2}$$

The available u is of order $\log s$ at the root of an s-wire contraction. Its recursive costs sum to $\exp(O(s/\sqrt{\log s}))$. Combining this with (1) produces the dense-input exponent. The candidate improvements in this note address these two steps explicitly.

3 An exact scalar parity replacement

3.1 Normalization and the local indicator

Write a normalized polynomial over $\mathbb{F}_2$ as

$$P_i(x) = c_i + \sum_{j=1}^{t_i} m_{ij}(x), \qquad c_i \in \{0,1\},$$

where each m_{ij} is a distinct nonconstant squarefree monomial of degree at most k. Repeated monomials have been canceled modulo two. Replace x_v in test i by the incidence variable z_{iv}, and write z_i for that test's variables. Define

$$E_i(z_i) = \frac{1 + (-1)^{c_i} \prod_{j=1}^{t_i} (1 - 2m_{ij}(z_i))}{2}. \tag{3}$$

The division is performed over the integers before reducing modulo a power of two. It is not inversion of 2 in $\mathbb{Z}/2^p\mathbb{Z}$.

Lemma 3.1 (Exact indicator)**.** *The expression* (3) *is an integer polynomial. For every Boolean assignment y to its incidence variables,*

$$E_i(y) = \mathbf{1}_{\{P_i(y)=0\}}.$$

Proof. The constant term of the numerator is $1 + (-1)^{c_i}$, which is even. Every nonempty selection of r factors from the product has coefficient $(-1)^{c_i+r}2^r$, also even. On Boolean inputs, each $m_{ij}(y)$ is zero or one, so $1 - 2m_{ij}(y) = (-1)^{m_{ij}(y)}$. Thus the product with the sign $(-1)^{c_i}$ equals $(-1)^{P_i(y)}$, and the claimed indicator follows. □

Let $\overline{E}_i$ denote the multilinear representative obtained by replacing each positive power of an incidence variable by its first power. It still agrees with E_i on the Boolean cube. For $S \subseteq I_i$, use $z_{i,S} = \prod_{v\in S} z_{iv}$ and write $c_{i,S} = [z_{i,S}]\overline{E}_i$.

For example, if $P = x_1x_2 + x_2x_3$, then

$$\overline{E} = 1 - x_1x_2 - x_2x_3 + 2x_1x_2x_3.$$

The cubic coefficient already records a factor of two. If instead the two monomials are disjoint cubic monomials A, B, the indicator is $1 - A - B + 2AB$. The coefficient at incidence degree six has valuation one. These are the elementary phenomena used in the general degree filtration.

3.2 Coefficientwise divisibility

We use $v_2(0) = +\infty$.

Lemma 3.2 (Local degree versus valuation)**.** *For every nonempty $S \subseteq I_i$,*

$$v_2(c_{i,S}) \geq \max\left\{0, \left\lceil \frac{|S|}{k} \right\rceil - 1\right\}. \tag{4}$$

The coefficient for $S = \varnothing$ is integral as well.

Proof. After dividing by two, a term selecting $r \geq 1$ monomials in (3) has coefficient $\pm 2^{r-1}$. Multilinearizing that term gives the union of the selected supports, of size at most kr. A contribution to the coefficient indexed by S therefore has $r \geq \lceil |S|/k \rceil$. Each contribution has the stated divisibility, so their sum does too. The statement for the empty support follows from Lemma 3.1. □

Work now in $R_p = \mathbb{Z}/2^p\mathbb{Z}$, and form

$$F(z) = \prod_{i=1}^{a} \overline{E}_i(z_i) \quad \text{in } R_p[z].$$

Different tests have disjoint incidence variables, so this product is multilinear. If a monomial uses support S_i in each test, its coefficient is exactly $\prod_i c_{i,S_i}$. No expansion over all supports is required to evaluate that particular coefficient.

Theorem 3.3 (Sharper incidence-degree cap)**.** *An incidence monomial of total degree d in F has coefficient divisible by*

$$2^{\max\{0,\lceil d/k\rceil - a\}}.$$

Consequently, modulo 2^p, every incidence monomial of degree greater than

$$K' = k(p + a - 1) \tag{5}$$

vanishes. For exact counting on n declared variables, take $p = n + 1$, giving $K' = k(n + a)$.

Proof. Let $b \le a$ be the number of nonempty supports S_i. Summing (4) over those supports gives at least

$$\sum_{S_i \neq \varnothing} \lceil |S_i|/k \rceil - b \ge \lceil d/k \rceil - a.$$

Valuations are also nonnegative. If $d > k(p+a-1)$, the last expression is at least p, so the coefficient is zero in R_p. □

The source bound is $k(a+2(p-1))$, whereas the new bound is $k(a+p-1)$ [2]. For the at-most-$6n$-test systems of the general reduction, these upper caps are respectively $8kn$ and $7kn$. For $k = 3$ this particular worst-case comparison is $24n$ versus $21n$. The exact improvement depends on the actual number of tests; these numbers are bounds, not the number of variables of a new SAT instance.

Proposition 3.4 (Sharpness in this representation)**.** *The cap $k(p+a-1)$ cannot be decreased for arbitrary such test systems merely by strengthening a universal coefficientwise degree argument.*

Proof. In test i, take $r_i \ge 1$ mutually disjoint monomials, all of degree k, and take no constant term. On independent incidence variables the coefficient selecting every one of these monomials in every test is

$$(-1)^{\sum_i r_i} 2^{\sum_i r_i - a}.$$

Choose $\sum_i r_i = p+a-1$. Its incidence degree is $k(p+a-1)$ and its valuation is $p-1$, so it survives modulo 2^p. Tests may reuse original variables; only their incidence copies are independent. The assertion concerns the representation, not a lower bound for solving the equations. □

4 Integrating the replacement into the exact counter

4.1 The counting identity

Identify the incidence copies by setting $z_{iv} = x_v$. By Lemma 3.1, over the integers,

$$N(\mathcal{P}) = \sum_{x \in \{0,1\}^n} \prod_{i=1}^{a} E_i(x) = \sum_x F((z_{iv} = x_v)_{iv}) \pmod{2^{n+1}}. \tag{6}$$

The count lies between zero and 2^n, so its least nonnegative residue modulo 2^{n+1} recovers the exact answer. Variables absent from every test remain declared; their free choices must be counted. Empty test lists and constant equations are handled before the main construction.

Use the source's test grouping, variable grouping, incidence-degree arrays, and subset indices, with the cap K' from Theorem 3.3. For a specified support S_i in each test, a test-group tensor has scalar entry

$$\prod_{i \in \mathcal{T}_g} c_{i,S_i} \pmod{2^{n+1}}. \tag{7}$$

For a variable group, count one factor of two for each variable whose incidence copies are all absent from the selected monomial. All other variables are forced to one when summing that monomial. Invalid binary subset indices receive entry zero. These are the same combinatorial counting rules as in the source; the coefficients are now scalar.

4.2 Coefficient evaluation without expanding the lift

The formal product in (3) can have exponentially many monomials. The intended entry routine does not construct that product.

Lemma 4.1 (Scalar coefficient oracle)**.** *For every $S \subseteq I_i$,*

$$c_{i,S} = \sum_{T \subseteq S} (-1)^{|S|-|T|} \mathbf{1}_{\{P_i(\mathbf{1}_T)=0\}}, \tag{8}$$

where variables outside T are set to zero. A coefficient can be computed in $2^{|S|}\,\mathrm{poly}(t_i, k, |I_i|, p)$ time and polynomial space, without V or the source's polynomials e_j.

Proof. For a multilinear polynomial G, evaluation at $\mathbf{1}_T$ is the sum of its coefficients indexed by subsets of T. Subset-lattice inversion gives (8), and Lemma 3.1 supplies its values. Evaluate the sparse polynomial over $\mathbb{F}_2$ for each T, add or subtract one, and retain one accumulator. No table of all subsets is needed. Its absolute value is at most $2^{|S|}$ before reduction modulo 2^p. □

The sparse-query inputs have $|I_i| \le k\lambda^c$ for a fixed c. Thus the coefficient oracle takes $2^{\lambda^{O_{k,c}(1)}}$ time, within the permitted $2^{O(s^{9/10})}$ local-entry bound when $s = n\nu$. The high-incidence branch and grouping bound the maximum degree by $n^{3/4}\lambda^{O_{k,c}(1)} \le s^{9/10}$ for sufficiently large n. All tensor descriptions remain polynomial in n.

Corollary 4.2 (Valid replacement in the source algorithm)**.** *Assuming the source's contraction theorem and preprocessing lemmas, replacing its dense-query lift by (3) gives a deterministic exact counter with polynomial space and the same proved leading arbitrary-density bound*

$$(n+m)^{O_k(1)} 2^{O_k(n \log\log n/\sqrt{\log n})}.$$

Its dense-query contractions can use $\ell = 1$ and scalar initial coefficients.

Proof. The preceding counting identity and tensor construction establish correctness. Equation (1) still holds with $K' = \Theta_{k,c}(n)$. Every other hypothesis of the source theorem is unchanged or weakened, and it explicitly permits $\ell = 1$. The source's exact sparsification and greedy parity-test selection then use this valid query oracle. □

5 Precisely what becomes faster or smaller

Let $p = n + 1$. The original dense-query coefficient ring is $(\mathbb{Z}/2^p\mathbb{Z})[V]/(V^p)$. The replacement starts in $\mathbb{Z}/2^p\mathbb{Z}$. The contraction proof still uses its unramified extension of degree e and working precision q; neither of those is removed.

The coefficient payload improves by a factor of p, and schoolbook multiplication in the removed axis improves by $\Theta(p^2)$. Fast polynomial multiplication would give a different arithmetic comparison. The whole program also stores graphs, mode tuples, interpolation data, and immutable entry descriptions, so a factor-n reduction in total space is not asserted. Likewise, no factor-n^2 reduction in total runtime follows just from the schoolbook multiplication count.

This replacement is *not* justified by substituting $V = 1$ into the old quotient ring. That substitution is not a ring homomorphism: $V^p = 0$ there, whereas $1^p = 1$. The new construction uses an

Quantity	Original lift	Scalar lift
Root coefficient payload	p^2 bits	p bits
One coefficient at extension degree e, precision q	eqp bits	eq bits
Coefficient products in one schoolbook V-multiplication	$p(p+1)/2$	1
Retained incidence-degree bound	$k(a+2p-2)$	$k(a+p-1)$
Local subset-evaluation output	A vector of p coefficients	One Boolean parity value
Readout after contractions	Coefficient-sum map after truncated products	Scalar residue

Table 2: Exact representation and subroutine comparisons. These are not implementation-independent speedup factors for the entire solver.

independent exact integer indicator and proves coefficientwise divisibility before reducing modulo 2^p. This removes the source's need to control a coefficient-sum map that does not preserve products.

An implementation should use (8) for entry lookups, retain the existing creation-precision and canonical-lift rules, and avoid storing the full expanded indicator. It may return zero immediately if $\lceil |S|/k \rceil - 1 \geq p$. It can multiply the per-test scalar coefficients sequentially. These changes are concrete even before any stronger factorization theorem is available.

6 Why several immediate exponent improvements fail

6.1 Incidence-support indexing has a real entropy cost

Take a tests, each with t disjoint nonconstant degree-k monomials, and no constant. On independent incidence variables,

$$\overline{E}_i \equiv 1 + P_i \pmod 2.$$

Their product has exactly $(t+1)^a$ distinct odd-coefficient incidence monomials. Each test independently contributes its constant or one of its t monomials; distinct incidence choices cannot coincide. Every one of these terms survives at every positive precision.

For $a = \Theta(n)$ and $t = (\log n)^c$, an explicit injective index of all these supports needs $\Omega(n \log\log n)$ bits. There are only $2^{O(\sqrt{n}\log n)}$ degree arrays in the source construction. By the pigeonhole principle, distributing the supports among those arrays leaves an array requiring $\Omega(n \log\log n)$ index bits. The tighter cap in Theorem 3.3 does not remove these odd terms.

This is a limitation of a representation, not of SAT. The same example with linear tests can be solved by Gaussian elimination. A successful improvement must use additional algebraic structure, combine incidence selections, or change the contraction method.

6.2 The weighted-partition estimate is sharp for its relaxation

Consider an abstract set with A modes of each weight $1, 2, \ldots$ and a budget P, in the regime $P/A \to \infty$. Let $q = \left\lfloor c\sqrt{P/A} \right\rfloor$ for a sufficiently small positive constant c. Choosing each of the Aq modes of weights at most q with exponent zero or one produces at least 2^{Aq} tuples. Even the tuple

choosing all of them consumes only

$$A\frac{q(q+1)}{2} < P.$$

Hence this relaxation contains $\exp(\Omega(\sqrt{AP}))$ tuples. Under the source scaling, their ordinary degree is only $Aq = \Theta(M/\sqrt{u}) \le M$. Imposing that ordinary-degree bound does not improve the exponent. Keeping an additional starting valuation of order $\log s$ is likewise insufficient when the important weights are of order $D/\sqrt{u} \gg \log s$.

This does not prove that every such tuple has a nonzero coefficient in every actual network. It shows why a better estimate of the currently enumerated relaxation alone is not a new algorithm.

6.3 Nonnegative Hermite tapers have a restricted limitation

Here is a more explicit reason to look beyond taper tuning. Let nonnegative multiplicities q_i vanish for $i \ge u$, and suppose their degree coverage is $\sum_{i<u} 2^{-i} q_i \ge cM$. Define

$$L_i = 2q_i - \sum_{j\ge 1} \frac{j}{2^j} q_{i+j}.$$

If $K = \max_i L_i$, then

$$q_i \le K/2 + \mathbb{E}[q_{i+J}], \qquad \Pr(J = j) = \frac{j}{2^{j+1}}.$$

Because $J \ge 1$, iterating reaches the zero boundary in at most $u - i$ steps. Thus $q_i \le K(u-i)/2$. Also $\max_i q_i \ge cM/2$, so $K \ge cM/u$.

For the finite grid of the source with $a \ge 2u$, truncating the sum at $j = a - i$ changes the loss by at most $O(u2^{-u}) \max_i q_i$. Combining this with $K \ge 2\max_i q_i/u$ preserves the $\Omega(M/u)$ lower scale for large u. This concerns the cardinal Hermite precision analysis with nonnegative multiplicities. It does not rule out a different recovery method that exploits dependencies between jets.

6.4 Other changes that do not certify a faster bound

Using the source clause network immediately after sparsification is weaker: for $k = 3$ its cost has exponent proportional to $n[\sigma + B/\sqrt{\log n}]$, where $B = O((\sigma^{-1}\log(2/\sigma))^2)$. Ignoring secondary logarithms, its best balance is $n/(\log n)^{1/6}$, worse than the source's dense bound.

Changing the counting modulus does not remove the dominant Hermite loss $O(MD/u)$. At an odd prime larger than the interpolation grid, denominators become units, but the binary geometric tile ratio no longer gains a power of that prime with each mode order. The existing truncation proof therefore cannot simply be reused. Replacing powers by a Newton basis introduces $v_2(b!) = b - O(\log b)$, which changes linear valuation growth only by a constant factor.

7 The stronger proposal: contract the mode sum again

The source already expresses a deformed contraction as

$$F(T) = \sum_{\beta:\,\sum_Z w_Z\beta_Z < P} \prod_{g=1}^{r} Z_{g,\beta} \pmod{2^P}, \tag{9}$$

where every mode has one source group and one target group. Instead of enumerating every β separately, encode bundles of such modes as edges of a new network. The index compression itself is valid. The missing factorization appears only after it.

7.1 One deformation stage with more groups

Start with a source-theorem input at ambient size s, precision $O(s)$, ambient degree bound $\Delta = \left\lceil s^{9/10} \right\rceil$ on the input maximum degree, and local entries evaluable in $\mathrm{poly}(s)\exp(O(s^{9/10}))$ time and polynomial space. For clarity, take polynomial-size descriptions, as in the SAT applications. Put

$$\lambda = \lceil \log_2 s \rceil, \quad r = \lceil \lambda \rceil, \quad u = \left\lfloor \frac{\lambda}{4096 \log_2 \lambda} \right\rfloor, \quad a = 2u, \quad L = 2^a, \quad D_{\mathrm{arr}} = r^{2a}. \tag{10}$$

These parameters are used for one stage, not for a recursively growing-group algorithm. If $M \le s/\lambda$, evaluate directly. Otherwise $M > s/\lambda$.

Writing $c = 1/4096$, we have

$$D_{\mathrm{arr}} = s^{4c+o(1)}, \qquad L = s^{o(1)}, \qquad 2D_{\mathrm{arr}}^2 \Delta = s^{9/10+8c+o(1)} = o(s/\lambda).$$

The source's other packing inequalities $M \ge LD_{\mathrm{arr}}$ and $2L \le \Delta$ also hold for large s. Its polynomial-time labeling and packet construction therefore remain available [3]. The child contraction for group g has at most $6M/r$ edges.

At an active interpolation node of valuation $i < u$, put $D = L/2^i$. The tapered precision and mode multiplicity estimates remain

$$P = O(MD/u), \qquad A_{\mathrm{total}} \le 6M/D, \qquad P \le s^{1+o(1)}. \tag{11}$$

Indeed $D \ge 2^{u+1}$ absorbs both the inherited $O(s)$ precision and the $O(Ma)$ recovery term. All parameters and all intermediate precisions are polynomial in s.

For fixed β, evaluate $Z_{g,\beta}$ by direct summation of its $O(s/r)$ internal bits. The original entries and the new packet-entry coefficients give the bound

$$E(s) = \mathrm{poly}(s)\exp(O(s/r + s^{9/10})), \tag{12}$$

with polynomial space. The packet-entry term $\exp(O(L \log s))$ is absorbed. This argument does not assume that a fast child contraction automatically works at the inflated precision.

7.2 Exact pair budgets and a shorter index network

Bundle modes by their unordered pair e of distinct source and target groups; retain their directions in the descriptions. Let A_e be a bound on the number of its modes at each weight, chosen so that $\sum_e A_e \le 6M/D$. Enumerate *exact* budgets $b_e \ge 0$ with $\sum_e b_e < P$. There are at most

$$(P+1)^{r(r-1)/2} = \exp(O((\log s)^3)) \tag{13}$$

arrays. Exact budgets are essential: using an upper bound for each pair and then summing over all bounds would count many tuples repeatedly.

Let $R_e(b_e)$ count the tuples in bundle e of exact weight b_e. For positive budgets and nonempty bundles, the weighted-partition estimate implies $\log R_e(b_e) = O(\sqrt{A_e b_e})$. A zero budget has one

tuple. Skip an impossible budget. Index a feasible fiber using $\lceil \log_2 R_e(b_e) \rceil$ bits, assigning zero to unused binary words. Then the number of index wires satisfies

$$\begin{aligned} W_{\text{array}} &\le r^2 + O\left(\sum_e \sqrt{A_e b_e}\right) \\ &\le r^2 + O\sqrt{\left(\sum_e A_e\right)\left(\sum_e b_e\right)} = O(M/\sqrt{u} + r^2) \\ &= O\left(s\sqrt{\frac{\log\log s}{\log s}}\right). \end{aligned} \tag{14}$$

Use a sufficiently large fixed constant to choose a common ambient upper budget

$$W = C_0 \left\lceil s\sqrt{\log\log s/\log s} \right\rceil.$$

Here W is an upper bound; some arrays have far fewer wires. Counting, ranking, and unranking weighted tuples are polynomial in s, because the numeric budget P, number of modes, and integer bit lengths are polynomial in s. Appendix A states a concrete dynamic program.

For each array, the new graph has one tensor per group, computing $Z_{g,\beta}$ from the decoded incident indices. Its contraction is exactly the part of (9) with that array. Summing arrays restores the whole expression. This exact reformulation, including invalid-index zeros, is the part tested by the second prototype.

7.3 The missing structural result

Unproved assumption 7.1 (Compact factorization of the group tensors, LF)**.** For the family of input networks under consideration, there is a uniform algorithm which, for every feasible first-stage interpolation instance and exact budget array, factors each group tensor as a Boolean tensor network such that:

(i) Each group tensor is preserved for every assignment to its boundary index bits, including zeros at invalid binary indices. The equality holds in the current coefficient algebra modulo 2^P.

(ii) The combined factorizations have $O(W)$ total edges, including all new internal edges, and maximum degree at most $\lceil W^{9/10} \rceil$. They have polynomially many vertices and polynomial-size descriptions.

(iii) Construction and every local factor-entry evaluation take at most $\text{poly}(s)\exp(O(s/r + s^{9/10}))$ time and polynomial space.

(iv) Entries are fixed at their creation precision P and admit the same canonical lifts used in the source. The original unramified coefficient field is retained.

A constant-factor subdivision of edges to make the result bipartite is allowed.

Assumption LF is not proved in this note or supplied by the source. For the SAT consequence, it suffices to prove it for the networks generated by the clause and sparse-query reductions. Assuming it for all source-theorem networks is a stronger research target.

The obstruction is concrete. The unfactored graph has only $r \asymp \log s$ group tensors. In the regime $W_{\text{array}} \asymp W$, some degree is at least $2W_{\text{array}}/r \gg W^{9/10}$. A short program for an entry is not a

factorization with small degrees. Restoring the original child networks retains about $2M$ original row and column wires in total, which also fails the required $O(W)$ edge budget.

Applying the existing packing argument directly to the high-degree graph does not give another $\sqrt{\log W}$ saving. If the second group alphabet has fixed size R, its packing condition is $W \geq 2R^{4a'}\Delta'$. Since $\Delta' \geq cW/r$, it permits only $a' = O(\log r)$ useful levels. Meanwhile the first stage has $u = O(\log s/\log r)$. Their product is only $O(\log s)$; the apparent two-stage saving returns to the original square-root-logarithm scale. LF must exploit extra structure of the group tensors, beyond index length.

8 Conditional running time and all remaining costs

8.1 A modest extension of the existing contraction analysis

The second contraction has initial precision $P = s^{1+o(1)}$, which may exceed its wire budget W. This requires an explicit check; the original initial precision hypothesis cannot be quoted unchanged.

Lemma 8.1 (Precision and entry-oracle extension)**.** *Let S be a size parameter and retain the source's constant $\epsilon = 1/(128 \cdot 1024)$. Its contraction proof extends to initial precision*

$$P_0 \leq S^{1+\epsilon/2},$$

in a fixed unramified coefficient ring of degree $O(\log S)$ with at least S^4 residue elements and polynomial-time arithmetic in its given integral basis, as in the source construction. A polynomial truncation length is permitted. If an original tensor entry takes $\mathrm{poly}(S)\exp(B(S))$ time and polynomial space, with fixed creation-precision semantics, the total time is

$$\mathrm{poly}(S)\exp\big(O(S/\sqrt{\log S}) + B(S)\big),$$

with polynomial space, under the other source size and degree hypotheses.

Proof. Let $\lambda = \lceil \log_2 S \rceil$ and use the source's parameters $u_d = \left\lfloor \epsilon\lambda/2^d \right\rfloor$ and $a_d = 2\sum_{j\leq d} u_j$. At an active root, $M > S/\sqrt{\lambda}$, and every retained node has $D \geq 2^{u_0+1} \geq cS^{\epsilon}$. Therefore

$$\frac{P_0}{MD/u_0} \leq O(\lambda^{3/2}S^{-\epsilon/2}) = o(1).$$

The needed bound $P_t = O(MD/u_0)$ still holds. At depths $d \geq 1$, $a_{d-1} \geq 2u_0$, so $S2^{a_{d-1}}$ absorbs the extra initial precision. The source's inherited-precision bound and subsequent argument then apply. Its original first invariant at depth zero is thus replaced, not silently assumed.

All precisions remain polynomial in S. Its branching and interpolation counts are unchanged. The cost of an original entry is a multiplicative factor, because descriptions are immutable and evaluated directly at their creation precision. Newly generated packet entries have their existing $\exp(O(S^{9/10}))$ bound, absorbed by the main exponent. The same depth-first traversal gives polynomial space. The proof's interpolation uses an adequate residue field, rather than requiring a freshly constructed extension at each application. Polynomial truncation length changes only polynomial arithmetic factors. □

In our application $P = W^{1+o(1)}$, so the lemma's explicit precision inequality eventually holds. Reuse the first field, whose degree is $4\lceil \log_2 s \rceil = O(\log W)$ and whose size is at least $s^4 > W^4$ for sufficiently large s. Tensoring two unrelated extensions and assuming the result is a field would not be a valid shortcut.

8.2 The conditional contraction theorem

Theorem 8.2 (Conditional second contraction)**.** *For a family satisfying Assumption 7.1, the source-theorem input networks can be contracted deterministically in polynomial space and time*

$$\operatorname{poly}(s)\, 2^{O(s\sqrt{\log\log s}/\log s)}. \tag{15}$$

Proof. Perform the one-stage deformation of Section 7, then apply LF to each exact-budget graph. By Lemma 8.1, its second contraction takes time

$$\operatorname{poly}(s) \exp\left(O\left(\frac{W}{\sqrt{\log W}}\right) + O(s/r + s^{9/10})\right).$$

Since $r = \Theta(\log s)$ and $W = O(s\sqrt{\log\log s/\log s})$, this is $\operatorname{poly}(s)\exp(O(s\sqrt{\log\log s}/\log s))$. The factor (13) and all polynomially many interpolation instances are smaller-order multiplicative overheads. Hermite recovery gives the original contraction. Process arrays, interpolation values, and recursive calls sequentially; every stage uses polynomial space. Explicit parameter checks and exhaustive handling of bounded exceptional sizes give a uniform conditional algorithm. □

8.3 Retuning preprocessing is necessary

Substituting $s = n\nu$ into (15) suggests the target $n\nu^{3/2}/\lambda$, but that substitution alone is incomplete. Two branches of the source use $2^{O(n/\sqrt{\lambda})}$ time. Either would eventually dominate the proposed bound if left unchanged.

Use the sparsification parameter

$$\sigma = 1/\lceil \log_2 n \rceil,$$

in place of order $1/\sqrt{\lambda}$. The source's sparsification lemma then gives $2^{O(n/\lambda)}$ branches and occurrence bound

$$B = O_k((\lambda \log \lambda)^{k-1}).$$

The sparse-check row lengths remain fixed powers of λ: the source has $P_{\text{hash}} = 20000B^2\mu^2$ and $D_0 = 320000B^2\mu^3$, with $\mu = O(\log n)$ after normalization. Candidate enumeration is still $2^{\lambda^{O_k(1)}}$, and every query still has at most $6n$ tests of degree at most k and polylogarithmic monomial count [1].

In the query reduction, also replace the high-incidence cutoff by

$$D_* = \max\{1, \lceil N_0/n \rceil\} \lceil \log_2 n \rceil, \tag{16}$$

where N_0 is the initial incidence count. At most n/λ variables exceed this cutoff. Fixing them costs $2^{O(n/\lambda)}$ branches. All remaining degrees are still polylogarithmic, the test and variable groups still have degree $n^{3/4}\lambda^{O_k(1)}$, and the wire budget is still $O_k(n\nu)$. The degree arrays contribute $2^{O_k(\sqrt{n}\log n)}$, also smaller than the target.

Corollary 8.3 (Conditional improvement for fixed-width SAT and counting)**.** *If LF holds for the networks arising from these reductions, exact counting and hence k-SAT are deterministic and polynomial-space with time*

$$(n+m)^{O_k(1)} 2^{O_k(n(\log\log n)^{3/2}/\log n)}.$$

For $m \le Dn$ with fixed D, the direct clause network gives

$$(n+m)^{O_{k,D}(1)} 2^{O_{k,D}(n\sqrt{\log\log n}/\log n)}.$$

Proof. The dense case uses $s = \Theta(n\nu)$ and the retuned preprocessing just verified. Its additional exponents n/λ, $\sqrt{n}\log n$, and $\lambda^{O_k(1)}$ are absorbed by $n\nu^{3/2}/\lambda$. At fixed density, the source clause network has $s = \Theta_{k,D}(n)$ and needs no dense reduction. Apply Theorem 8.2. □

These are conditional mathematical implications, not certified new running times. The needed construction is LF. It would be a substantial structural result, and no claim that it is true follows from its usefulness.

9 A narrower alternative: aggregate equal variable unions

There is a separate target for removing only the dense-case $\log\log n$. In the scalar polynomial $F(z)$, identify all copies of variable v and reduce by $x_v^2 = x_v$. For $S \subseteq [n]$ define

$$c_S = \sum_{\substack{Q \text{ an incidence support} \\ \operatorname{vars}(Q)=S}} [z_Q]F(z) \pmod{2^{n+1}}.$$

Then

$$N(\mathcal{P}) \equiv \sum_{S\subseteq[n]} 2^{n-|S|}c_S \pmod{2^{n+1}}. \tag{17}$$

This follows by summing each resulting squarefree monomial over the Boolean cube. There are only 2^n possible unions, even when the incidence expansion has $2^{\Theta(n\log\log n)}$ supports.

If the functional (17) can be uniformly compiled into an $O(n)$-wire network of maximum degree at most $n^{9/10}$, with polynomial descriptions and $2^{O(n^{9/10})}$-time, polynomial-space entry routines, the existing contraction theorem gives

$$(n+m)^{O_k(1)}2^{O_k(n/\sqrt{\log n})}.$$

The source's original preprocessing costs fit that target. This proposal does not require the improved engine, but it also lacks a compiler theorem.

The coefficient identity is not that theorem. An n-port tensor for c_S has forbidden degree, and directly evaluating a coefficient by a subset sum can require $2^{|S|}$ work. The prototype's dictionary of union supports is an exponential reference calculation for small inputs. Its correctness does not certify a fast compiler. A useful next investigation would seek a factorization that preserves the quotient $x_v^2 = x_v$ without explicitly retaining all test–variable choices.

10 Implementation artifacts and checks

10.1 Scalar parity prototype

`code/parity_lift.py` represents a multilinear polynomial by a dictionary from support bit masks to integer coefficients. Its reference multiplication uses support union, reflecting $x_v^2 = x_v$. It implements normalization, the exact parity product, direct coefficient inversion, the old truncated-V comparison, and independent exhaustive common-zero counts. Only the coefficient oracle is intended as a replacement primitive; expanding whole dictionaries is used to validate small examples.

The executed suite, with seed 20261026, passed:

Check	Cases
Exhaustive normalized polynomials	2,304
Truth-table comparisons	34,816
Coefficient and divisibility comparisons	34,816
Random systems compared with assignment enumeration	300
Old truncated-V products versus scalar products	250
Global incidence-degree checks	120
Sharp degree-cap examples	12

Constants, empty equations, unused variables, and cancellation of repeated monomials are included. The old-versus-new test checks products before the source readout, rather than assuming that summing coefficient vectors is a ring homomorphism.

10.2 Weighted-mode indexing prototype

`code/weighted_partitions.py` counts, ranks, and unranks nonnegative exponent tuples of prescribed exact weight. It also checks a three-group mode sum by comparing direct enumeration with exact pair-budget arrays and binary indices. Invalid padding words have weight zero in the encoded sum. The toy group functions contain nonlinear terms and depend only on incident modes, so the test exercises the relevant factorization of the summation.

The executed suite passed 40 weight lists, 360 exact fibers, 1,525 exact rank round trips, 400 cumulative budgets, 5,065 cumulative round trips, 360 invalid-rank checks, and 12 modular grouped-sum identities. At strict budget six, the direct sum has 297 tuples; the encoded calculation has 297 valid rank words across 462 exact budget arrays and five invalid padding words. Results agree modulo both 97 and 101. These small moduli are test arithmetic only; the proposed contraction still uses the source's two-adic coefficient ring.

10.3 What was and was not validated

The tests validate exact identities and the finite indexing construction. They do not validate LF, the asymptotic contraction implementation, or a new SAT runtime. The elementary proofs establish the general algebraic claims; finite tests are supplementary. The source theorem is an external dependency, and the reported formalization scope is attributed to the repository.

The archive contains the self-contained LaTeX source and compiled PDF, both standard-library prototypes, recorded JSON results, a source manifest, a machine-readable research-status file, and build/verification scripts. Run

```
python3 code/parity_lift.py
python3 code/weighted_partitions.py
python3 verify_package.py
python3 build_report.py
```

The final command requires a TeX installation. Timing fields in the JSON describe the small validation suite and are not benchmarks of a SAT solver.

11 Recommended next mathematical step

The scalar replacement can be implemented and formalized independently of the speculative compression work. Its central proof obligations are short: the parity product identity, support-degree divisibility, coefficient inversion, and the existing tensor-network counting identity. It also removes the auxiliary coefficient-sum map from the dense-query proof interface.

For the requested leading exponent improvement, the most specific next task is to construct or refute LF on the group tensors generated by sparse degree-three parity queries. The construction must expose every auxiliary wire, count their total, and provide bounded-degree local entries with fixed creation-precision semantics. An equality of formal sums or a fast oracle for one large tensor is insufficient. Small exact tensor-rank experiments may help choose a representation, but do not replace these size and uniformity bounds.

If LF fails on that family, the union-support compiler in Section 9 is a distinct route to a smaller improvement. Either success would bypass a demonstrated representation cost. Until such a result is proved, the strongest established conclusion of this report remains the scalar arithmetic refinement, with the source's leading exponent unchanged.

A Exact weighted-tuple indexing

For positive weights $w_0, \dots, w_{q-1}$ and an integer maximum total B, let $D_i(b)$ count tuples on coordinates $i, \dots, q-1$ of exact weight b. Use boundary values $D_q(0) = 1$ and $D_q(b) = 0$ for $b > 0$. Then

$$D_i(b) = D_{i+1}(b) + \begin{cases} D_i(b - w_i), & b \geq w_i, \\ 0, & b < w_i. \end{cases}$$

Compute i in decreasing order and b in increasing order. This uses $O(qB)$ integer cells and arithmetic operations; it is pseudopolynomial in the numeric budget. In the application q and $B = P$ are polynomial in s, so it is polynomial in the input size parameter used in the proof.

For lexicographic ranking in an exact fiber, choosing the next exponent j leaves $D_{i+1}(b - jw_i)$ completions. Sum these counts for all smaller choices. Unranking selects the unique interval containing the rank. Cumulative strict-budget indexing first chooses the total $b < P$ using the counts $D_0(b)$ and then uses the exact-fiber procedure. A binary word whose integer value is at least the fiber size is invalid. This gives a bijection, rather than an encoding that accidentally counts several upper budgets for one tuple.

B Source locations for an implementation patch

The source uses labels rather than stable theorem numbers across revisions. These labels identify the relevant locations at the pinned commit:

Source label or file	Relevance
`lem:rational-idempotent`, `lem:total-order`	The old lift and auxiliary V-truncation; replaced only in the quantitative dense-query route.
`lem:tensor-sparse-query`	Install the scalar coefficient oracle and counting identity here.
`eq:sharp-incidence-degree`	Replace the cap by $k(n+a)$.
`eq:sharp-wire-budget`	Keep the subset-index proof with the new cap; its asymptotic order is unchanged.
`eq:tensor-precision-bound`	Preserve canonical lifts; extend the root analysis only for the conditional second contraction.
`eq:tensor-mode-multiplicity`, `eq:tensor-partition-count`	The weighted-mode index bounds used in the proposed compression.
`eq:tensor-recursion-identity`	The sum regrouped by exact pair budgets.
`lem:sparsify`, `lem:sparse-checks`, `lem:greedy-hashes`	Retune sparsification to $1/\lambda$ only for the conditional stronger bound.
`lean/docs/378.md`	The repository's stated formalization scope, weaker than the revised quantitative manuscript.

References

[1] OpenAI, *The Exponential Time Hypothesis is False*, October 17, 2026, revised October 25, 2026. Inspected at repository commit `7ef5c0cc5ca4b0e072b7a050fca09ab24bc6fa3a`. Pinned main LaTeX source.

[2] OpenAI, *A sharper bound for exact counting*, quantitative reduction appendix of [1]. Pinned quantitative-reduction.tex.

[3] OpenAI, *Proof of the contraction theorem*, quantitative contraction appendix of [1]. Pinned quantitative-contraction.tex.

[4] OpenAI, *The Exponential Time Hypothesis is False: Lean formalization scope*, repository documentation, inspected at the same commit. Pinned scope statement.